

APPLICATION OVERVIEW

Croar Payroll

A multi-tenant payroll management platform — define salary structures, run payroll cycles, and issue payslips through a guarded, permission-controlled lifecycle.

Stack: FastAPI (async) · Next.js + Tailwind · PostgreSQL

Date: 17 June 2026

1. Executive Summary

Croar Payroll is a decoupled single-page application backed by a JSON API. The Next.js client handles presentation; the FastAPI backend owns every business rule, calculation, and persistence concern. The system is **multi-tenant** — every record is scoped to a company — and protected by **JWT authentication with role-based access control**.

Its correctness-critical core is mature: array-based salary structures, a deterministic `Decimal` calculation engine, statutory snapshots, an audited approval workflow, and payslips that are released only when a cycle is fully paid. Recent work has extended it toward a managed-service feature set: per-run adjustments, CSV/PDF reports, an audit trail, an income-tax (TDS) engine, and an org-settings module.

Multi-tenant by design
Every query is scoped to the signed-in user's company.
Tenants only ever see their own data.

Secure by default
JWT auth + fine-grained `payroll:*` permissions enforced on the API and mirrored in the UI.

Auditable
Append-only audit log records who did what and when; activity is browsable in-app.

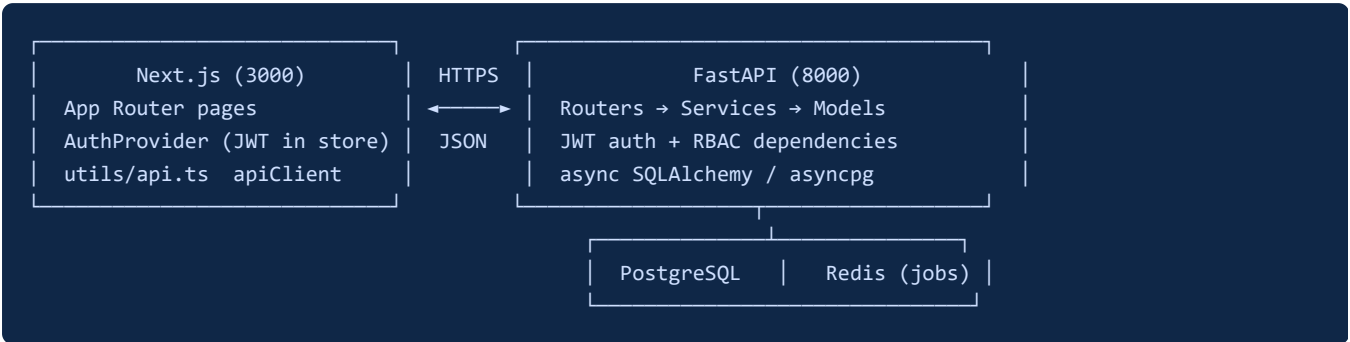
Correct money math
Pure `Decimal` engine, rounded per line (`ROUND_HALF_UP`) to avoid cent drift.

2. Technology Stack

Layer	Technology
Backend	FastAPI 0.115, Python 3.10+, async SQLAlchemy 2.0, Pydantic v2
Database	PostgreSQL (asyncpg driver), Alembic migrations
Auth	PyJWT bearer tokens, password hashing, RBAC
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4
Async / jobs	Celery + Redis (scaffolded)
Serving	Uvicorn / Gunicorn
Tooling	pytest + coverage, ruff, mypy, pre-commit, Docker Compose

3. Architecture

A decoupled SPA + API. The Next.js client talks to FastAPI over JSON; the backend is layered so each tier has a single responsibility.



Backend layering

- **Routers** stay thin — validate input, declare the required permission, delegate.
- **Dependencies** handle `get_current_user`, company scoping, and `require_permission`.
- **Services** hold testable business logic — e.g. `compute_payslip` is pure math with no DB dependency.
- **Models** are SQLAlchemy ORM with constraints and soft-delete columns.

4. Data Model

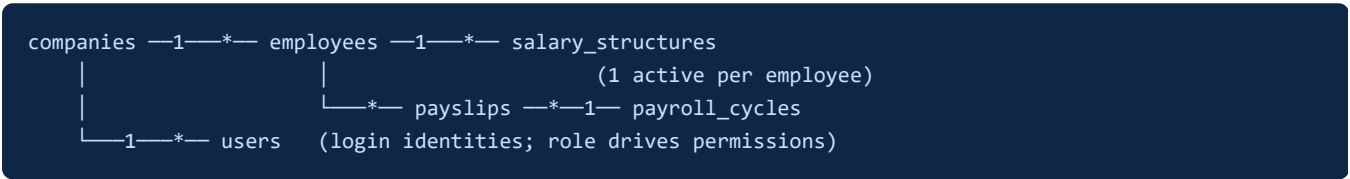
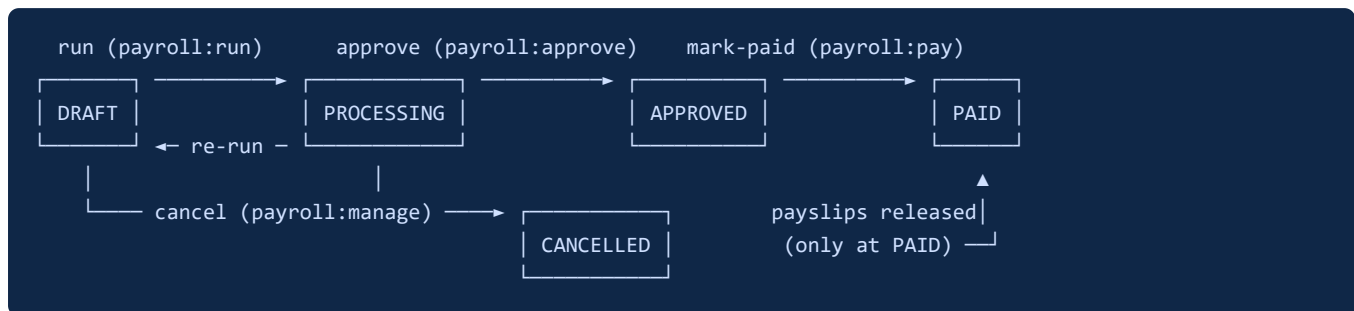


Table	Purpose
companies	Tenant root; every row carries <code>company_id</code> .
users	Login identities with a role (ADMIN / HR / VIEWER).
employees	People to pay; soft-deletable; unique email per company.
salary_structures	Earnings + deductions money lines, CTC, <code>lop_days</code> ; one active each.
payroll_cycles	A pay period with <code>status</code> and rolled-up totals.
payslips	One per employee per cycle; immutable snapshot of resolved lines.

Conventions: UUID primary keys · audit columns (`created_at` , `updated_at` , `deleted_at`) on every table · soft delete via `deleted_at IS NULL` · statuses stored as strings, validated through Python enums.

5. Payroll Lifecycle

A cycle moves through a guarded state machine. Each transition requires a specific permission and is rejected (409) from any other state.



- **Run** is allowed on DRAFT or PROCESSING (re-run recalculates idempotently in place).
- **Cancel** is allowed from any non-PAID state.
- A **PAID cycle is terminal** — it cannot be re-run, cancelled, or deleted.
- **Payslips are visible only once the cycle is PAID.**

6. Roles & Permissions

Permissions are fine-grained `payroll:*` capabilities; roles bundle them. The API enforces them via a `require_permission(...)` dependency (403 on failure); the frontend hides disallowed nav and blocks direct navigation.

Permission	Meaning	ADMIN	HR	VIEWER
<code>payroll:read</code>	View cycles, structures, payslips, staff	✓	✓	✓
<code>payroll:configure</code>	Create/edit structures, employees, cycles	✓	✓	—
<code>payroll:run</code>	Run / recalculate a cycle	✓	✓	—
<code>payroll:approve</code>	Approve a processed cycle	✓	✓	—
<code>payroll:pay</code>	Mark a cycle paid	✓	✓	—
<code>payroll:manage</code>	Cancel / delete cycles	✓	✓	—
<code>users:manage</code>	Create / list users	✓	—	—

7. Payslip Calculation

`compute_payslip` (in `app/services/payroll_service.py`) is pure, deterministic `Decimal` math:

- **Resolve earnings** — each line is `fixed`, a `percent` of another line's code, or a percent of gross-so-far when `percent_of` is omitted.
- **Apply LOP proration** — `multiplier = (working_days - lop_days) / working_days` on a fixed 30-day basis; earnings are pro-rated.
- **Resolve deductions** — same line rules, evaluated against pro-rated earnings; statutory + estimated TDS where enabled.
- **Net pay** = gross – total deductions.

Every resolved line is rounded to 2 dp (ROUND_HALF_UP) before summing to avoid cent drift. LOP greater than the working-days basis reports that employee in the run's `skipped` list rather than failing the whole run.

8. Feature Highlights

Employees

Full CRUD, unique email per company, soft delete that preserves payslip history and deactivates structures.

Salary Structures

One active per employee, array-based earnings/deductions, LOP, live monthly estimate while editing (server-computed).

Payroll Cycles

Idempotent runs, employees without a structure surfaced in a `skipped` list, per-cycle roll-up totals.

Per-run Adjustments

One-time bonus / arrears / deductions applied on a cycle, editable only in DRAFT/PROCESSING.

Reports & Audit

CSV + PDF salary-register and payroll-summary exports; append-only audit trail with an in-app activity page.

Taxes & TDS

Employee IT declarations (both regimes), an opt-in income-tax estimator, and TDS challan records.

Org Settings

Editable organisation profile (legal name, PAN/TAN, address) — read for all, write for admins.

Dashboard

Company-wide overview: coverage, structure counts, cycle status, disbursed vs. pending — one server call.

9. Selected API Surface

Base URL `http://localhost:8000` · interactive docs at `/api/docs` . All `/api/v1/...` endpoints require a bearer token (except `auth/login`).

Method	Path	Permission	Description
POST	<code>/auth/login</code>	public	Email/password → JWT + user
GET	<code>/payroll/dashboard</code>	payroll:read	Company overview
POST	<code>/payroll/structures</code>	payroll:configure	Create salary structure
POST	<code>/payroll/cycles/{id}/run</code>	payroll:run	Run / recalculate cycle
POST	<code>/payroll/cycles/{id}/approve</code>	payroll:approve	PROCESSING → APPROVED
POST	<code>/payroll/cycles/{id}/mark-paid</code>	payroll:pay	APPROVED → PAID
GET	<code>/payroll/cycles/{id}/payslips</code>	payroll:read	List payslips (PAID only)

10. Roadmap

Croar's core is benchmarked against RazorpayX and Zoho Payroll. The correctness-critical engine is complete; remaining work is largely the managed-service layer.

Delivered DONE

- Per-run adjustments / off-cycle bonus runs
- Reports & registers (CSV / PDF)
- Audit / activity log
- TDS / income-tax estimator (both regimes, opt-in)
- Organisation settings & tax forms module

Buildable next PLANNED

- Form 16 / 24Q generation, attendance→LOP automation & leave
- Employee self-service portal (new EMPLOYEE role scoped to own data)
- Loans/advances, flexible benefits, reimbursements, salary revision & arrears, full & final settlement
- Departments/designations, multi-level approvals, notifications

Needs external integration BLOCKED

- Real bank/UPI disbursement (payout API)
- Automated government filing portals (challan/return files can still be generated offline)
- Live attendance check-in/out & HRMS/accounting connectors